Simulated Annealing

1 WORK DURING THE LAB

1. Implement the algorithm **Simulated Annealing** for *TSP*.
2. Test the algorithm for the same two problem instances selected in the previous lab, considering different parameter settings.

Points for the work during the lab: **25p**

2 ASSIGNMENT A4

1. Implement the algorithm **Simulated Annealing** for the *knapsack problem*.
2. Perform experiments for the two problem instances available (size 20 and 200) and compare results, considering different parameter settings for the algorithm.

Deadline to submit A4: **Lab 5**

Points for A4: **25p**

Late submission of A4 in Lab 6 will be penalised by 15 points. No later submissions are allowed.

3 REQUIREMENTS

1. Source code (notebook) needs to be documented.
2. Algorithms have to be tested for several parameter values (sufficient to clearly determine performance).
3. Experiments must be performed for all available problem instances and results compared for diferent parameter settings.
4. Results of the experiments need to be saved in output files, indicating solution quality, parameter values used, number of runs.
5. A report should capture the following: problem definition, algorithm used (name, steps/pseudocode), parameter setting, comparative results of experiments, discussion of results.

4 SIMULATED ANNEALING

- Function *g*:
  - T(k+1)= alpha * T(k), where alpha is in [0,1] closer to 1 (e.g. alpha=0.99)
  - T(k+1)=T(k)/log(k+1)
  - T(k+1)=T(k)/k

  where T(k) is the temperature at iteration k.

- Initial value of temperature should be high enough ex. T=10000.

- Termination condition / halting criterion:
  - Temperature reaches a minimum predefined temperature value
  - Number of iterations reaches a maximum value
  - Evaluation function reaches a certain value

```
begin
    t = 0
    initialize T
    select a current point c at random
    evaluate c
    repeat
        repeat
            select a new point x from the neighborhood of c
            if eval(c) < eval(x)
                then c ← x
                else if random[0,1) < e^((eval(x)−eval(c))/T) then c ← x

        until (termination-condition)
        T ← g(T, t)
        t ← t + 1
    until (halting-criterion)
end
```